

United International University



Course Title: Database Management System Lab

Course Code: CSE 3522

Section: B

TERM PAPER

Design and Implementation of a Relational Database Solution for a Micro Finance Loan Management System

A comprehensive database engineering approach utilizing advanced data modeling, schema optimization, transactional integrity, and analytical reporting in the microcredit landscape of Bangladesh.

Submitted By	Submitted To
Name: Gaffar Kadir ID: 0112330096 Department: Department of CSE Institution: United International University	Md. Golam Hossain Lecturer Department of CSE United International University

Table of Contents

1. Introduction	3
1.1 Background of Microfinance in Bangladesh	3
1.2 Problem Statement	3
1.3 Objectives of the Database Solution	3
1.4 Scope of the Project	3
2. Business Process & Requirements Specification	4
2.1 Core Business Operations and Life Cycle	4
2.2 User Roles, Operations, and Permissions	4
3. Data Modeling & Schema Design	5
3.1 Entity-Relationship Diagram (ERD)	5
3.2 Detailed Data Dictionary (Core & Transaction Tables)	6
3.3 Relational Schema and Normalization Analysis	8
4. Database Schema Implementation	9
4.1 SQL Table Creation Script (DDL)	9
4.2 Realistic Seed Data Insertion (DML)	10
5. Advanced Database Concepts	11
5.1 Indexing Strategy and Query Optimization	11
5.2 Database Partitioning Strategy	11
5.3 Triggers and Business Rules Enforcement	12
5.4 Transaction Management and Concurrency Control	13
6. Complex SQL Queries & Analytical Reporting	14
6.1 Portfolio at Risk (PAR) Analysis	14
6.2 Collection Efficiency Rate (CER) per Branch	14
6.3 Top Field Officers Performance Ranking	15
6.4 Client Savings vs. Outstanding Loan Balance Summary	15
6.5 Monthly Disbursement and Recovery Trend Analysis	15
7. Limitations & Discussion	16
8. Conclusion & Recommendations	16
9. References (APA Style)	16

1. Introduction

1.1 Background of Microfinance in Bangladesh

Microfinance has been a cornerstone of socio-economic development and poverty alleviation in developing nations, with its origins deeply rooted in Bangladesh. Pioneers like Nobel Laureate Professor Muhammad Yunus and organizations such as Grameen Bank, BRAC, and ASA revolutionized the financial sector by introducing microcredit. This model provides small, collateral-free loans to low-income individuals, primarily rural women, enabling them to engage in self-employment and income-generating activities.

Unlike commercial banking, the operational framework of microfinance relies heavily on group-based lending (the joint-liability model) and weekly recovery schedules. Borrowers form small groups (typically five members) and larger centers. The group members co-guarantee each other's loans, creating social collateral that replaces physical assets. This unique approach has sustained repayment rates exceeding 95% in Bangladesh.

1.2 Problem Statement

Despite the massive reach of Micro Finance Institutions (MFIs) in Bangladesh, many rural branches still operate using paper-based ledger books or fragmented spreadsheet applications. These legacy systems suffer from significant operational inefficiencies: manual calculation errors in interest and schedules, ledger inconsistencies, duplicate records, vulnerability to data loss, and delays in generating consolidated reports.

Furthermore, managing complex operations—such as tracking weekly loan repayments, calculating interest portions of payments, managing mandatory savings ledgers, enforcing joint-guarantor compliance rules, and monitoring the Portfolio at Risk (PAR) in real-time—is extremely difficult without a centralized, transaction-safe database.

1.3 Objectives of the Database Solution

The main objective of this project is to design and implement a centralized, secure, normalized, and transaction-safe Relational Database Management System (RDBMS) tailored to MFI operations. The database will enforce critical business rules, automate schedules, ensure data integrity through ACID compliance, and support analytical reporting for branch managers and executives.

1.4 Scope of the Project

The project covers structural entities from MFI branches down to individual clients, details on loan packages (duration, flat interest rates, frequency), savings account structures (deposits and withdrawals), loan repayment tracking (weekly schedules, installment processing), and complex analytical queries for risk assessments.

2. Business Process & Requirements Specification

2.1 Core Business Operations and Life Cycle

The database must support the operational lifecycle of a Micro Finance Institution:

- 1. Branch & Staff Setup:** Each branch is created with details. Field Officers are hired and assigned to specific branches to manage centers.
- 2. Center & Group Formation:** Field Officers establish Centers (geographical meeting spots, e.g., in rural villages) and define weekly meeting days (e.g., every Tuesday). Within each center, borrowers form Groups of exactly five members.
- 3. Client Admission:** Clients submit their details (demographics, National ID card number, and contact info). They are assigned to a Group, which automatically links them to a Center and a Branch. A mandatory savings account is opened for them immediately.
- 4. Loan Application & Appraisal:** A client selects a Loan Product (e.g., General Loan, Agriculture Loan) and applies. The system checks if the client has any outstanding active loans; new loans are only permitted if all prior loans are fully settled.
- 5. Amortization Schedule Generation:** Upon loan approval, an installment plan is auto-generated based on the loan duration and repayment frequency, calculating the precise principal and flat interest due per installment.
- 6. Weekly Center Meetings & Collections:** On the designated meeting day, the Field Officer visits the center, collects the scheduled loan installments, and collects mandatory weekly savings. These cash amounts are entered into the database.

2.2 User Roles, Operations, and Permissions

User Role	Permitted Database Operations	Business Purpose
System Admin	Full access (SELECT, INSERT, UPDATE, DELETE) on all tables; DDL operations.	Database maintenance, schema updates, user management, and archiving.
Branch Manager	SELECT on all tables; INSERT/UPDATE on staff, center, and client registration. Approves loans.	Branch oversight, employee productivity tracking, and credit risk management.
Field Officer	SELECT on clients/groups; INSERT on Repayments and SavingsTransactions.	Recording daily collections at center meetings and verifying client account balances.

3. Data Modeling & Schema Design

3.1 Entity-Relationship Diagram (ERD)

The relational structure represents a group-lending paradigm. The diagram below shows the 11 entities and their cardinality rules:

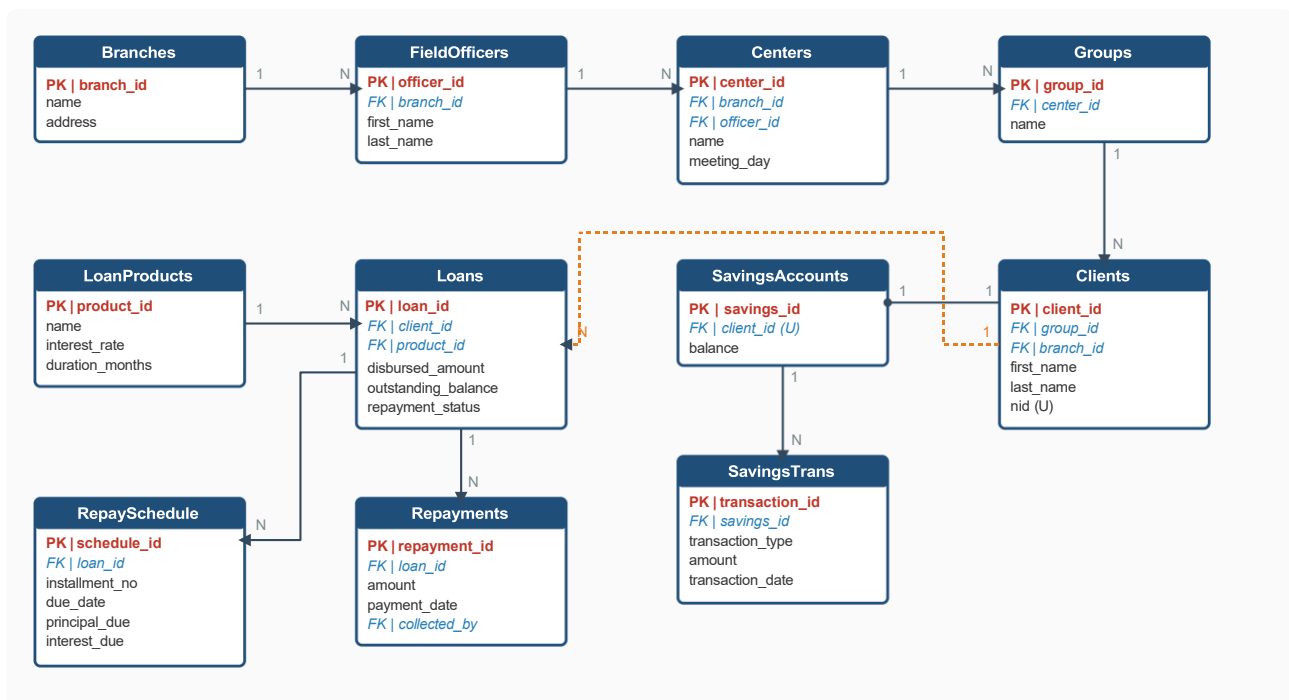


Figure 3.1: Entity-Relationship Diagram (ERD) with relational mappings and structural constraints.

3.2 Detailed Data Dictionary (Core & Transaction Tables)

The database dictionary provides structural metadata about all tables.

Table 1: Branches

Column Name	Data Type	Constraints	Description
branch_id	SERIAL	PRIMARY KEY	Unique identifier for each physical branch office.
name	VARCHAR(100)	NOT NULL, UNIQUE	Geographical branch name (e.g., 'Savar Branch').
address	TEXT	NOT NULL	Detailed location address of the branch.
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Date and time when the branch record was created.

Table 2: FieldOfficers

Column Name	Data Type	Constraints	Description
officer_id	SERIAL	PRIMARY KEY	Unique identifier for each field staff member.
<i>branch_id</i>	INT	REFERENCES Branches(branch_id)	The branch the field officer is currently assigned to.
first_name	VARCHAR(50)	NOT NULL	First name of the staff member.
last_name	VARCHAR(50)	NOT NULL	Last name of the staff member.
hire_date	DATE	NOT NULL, CHECK (hire_date <= CURRENT_DATE)	Employment start date.
status	VARCHAR(20)	CHECK (status IN ('Active', 'On Leave', 'Terminated'))	Current status of the field officer (Default: 'Active').

Table 3: Centers

Column Name	Data Type	Constraints	Description
center_id	SERIAL	PRIMARY KEY	Unique identifier for the geographic center.
<i>branch_id</i>	INT	REFERENCES Branches(branch_id)	The branch office supervising this center.
<i>officer_id</i>	INT	REFERENCES FieldOfficers(officer_id)	The Field Officer responsible for weekly collections here.
name	VARCHAR(100)	NOT NULL	Name of the center meeting place (e.g., 'Modhumoti Center').
meeting_day	VARCHAR(15)	CHECK (meeting_day IN ('Saturday', 'Sunday', ...))	Fixed weekly weekday when the center meetings are held.

Table 4: Groups

Column Name	Data Type	Constraints	Description
group_id	SERIAL	PRIMARY KEY	Unique group identifier.
<i>center_id</i>	INT	REFERENCES Centers(center_id)	The parent center this group belongs to.
name	VARCHAR(50)	NOT NULL	Name of the group (e.g., 'Shapla Group').

Table 5: Clients

Column Name	Data Type	Constraints	Description
client_id	SERIAL	PRIMARY KEY	Unique identifier for the microfinance borrower.
<i>group_id</i>	INT	REFERENCES Groups(group_id)	Joint-liability group the client is member of.
<i>branch_id</i>	INT	REFERENCES Branches(branch_id)	Branch which serves the client.
first_name	VARCHAR(50)	NOT NULL	First name of the client.
last_name	VARCHAR(50)	NOT NULL	Last name of the client.
nid	VARCHAR(20)	NOT NULL, UNIQUE	National Identification Number (crucial check).
admission_date	DATE	NOT NULL, DEFAULT CURRENT_DATE	The date the client joined the MFI.
status	VARCHAR(20)	CHECK (status IN (‘Active’, ‘Suspended’, ‘Closed’))	Status of MFI client account (Default: ‘Active’).

Table 6: LoanProducts

Column Name	Data Type	Constraints	Description
product_id	SERIAL	PRIMARY KEY	Unique identifier for the loan product.
name	VARCHAR(100)	NOT NULL, UNIQUE	Name of the loan scheme (e.g., ‘Microenterprise Loan’).
interest_rate	DECIMAL(5,2)	NOT NULL, CHECK (interest_rate >= 0.00)	Annual flat interest rate (e.g., 12.50%).
duration_months	INT	NOT NULL, CHECK (duration_months > 0)	Duration of the loan program (e.g., 12 months).

Table 7: Loans

Column Name	Data Type	Constraints	Description
loan_id	SERIAL	PRIMARY KEY	Unique identifier for this loan instance.
<i>client_id</i>	INT	REFERENCES Clients(client_id)	Borrower who received the loan.
<i>product_id</i>	INT	REFERENCES LoanProducts(product_id)	The category structure of the loan.
disbursed_amount	DECIMAL(12,2)	NOT NULL, CHECK (disbursed_amount > 0.00)	Total principal disbursed to the client.
outstanding_balance	DECIMAL(12,2)	NOT NULL, CHECK (outstanding_balance >= 0.00)	Remaining amount (principal + interest) left to pay.
repayment_status	VARCHAR(25)	CHECK (repayment_status IN (‘Active’, ‘Paid’, ...))	Current collection phase (Default: ‘Active’).

3.3 Relational Schema and Normalization Analysis

Database design must eliminate redundancy and avoid modification anomalies (Insert, Update, Delete). The proposed relational schema has been subjected to standard database normalization rules.

First Normal Form (1NF) Compliance

A relation is in 1NF if and only if all attribute values are atomic and there are no repeating groups. All tables comply with 1NF. For instance, in the `Clients` table, demographics such as `first_name`, `last_name`, and `nid` are stored in separate, single-valued columns. Phone numbers, which could be multiple, are restricted to a single primary contact per client in this entity, or placed in an external table if multiple are required.

Second Normal Form (2NF) Compliance

A relation is in 2NF if it is in 1NF and every non-prime attribute is fully functionally dependent on the primary key (no partial dependencies). In our design, all primary keys are single-column identifiers (e.g., `client_id`, `loan_id`, `repayment_id`). Since there are no composite primary keys in the core tables, partial functional dependencies are mathematically impossible, satisfying 2NF.

Third Normal Form (3NF) Compliance

A relation is in 3NF if it is in 2NF and no non-prime attribute is transitively dependent on the primary key. Let's analyze a potential violation: if we stored the `meeting_day` of a center directly inside the `Clients` table, we would have a transitive dependency:

$$client_id \rightarrow group_id \rightarrow center_id \rightarrow meeting_day$$

If a center changed its meeting day, we would have to run an UPDATE query across hundreds of clients, risking data inconsistency. To prevent this, we normalized the relations:

- `Clients` contains only `group_id`.
- `Groups` contains `center_id`.
- `Centers` contains `meeting_day` and `officer_id`.

By segregating these relationships into separate entities, all transitive dependencies are broken, ensuring 3NF.

Boyce-Codd Normal Form (BCNF) Analysis

A relation is in BCNF if for every non-trivial functional dependency $X \rightarrow Y$, X is a superkey. In our tables, the only non-trivial dependencies are those where the determinant is a primary key (e.g., `nid` $\rightarrow$ `client_id` in the `Clients` table). Since National ID (`nid`) is a unique attribute and serves as a candidate key, this dependency satisfies BCNF. Thus, the database is fully optimized and safe from anomalies.

4. Database Schema Implementation

4.1 SQL Table Creation Script (DDL)

The SQL DDL script written below targets PostgreSQL, creating all tables with appropriate column constraints:

```
-- Core Setup
CREATE TABLE Branches (
    branch_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    address TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE FieldOfficers (
    officer_id SERIAL PRIMARY KEY,
    branch_id INT NOT NULL REFERENCES Branches(branch_id),
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    hire_date DATE NOT NULL CHECK (hire_date <= CURRENT_DATE),
    status VARCHAR(20) DEFAULT 'Active' CHECK (status IN ('Active', 'On Leave', 'Terminated'))
);

CREATE TABLE Centers (
    center_id SERIAL PRIMARY KEY,
    branch_id INT NOT NULL REFERENCES Branches(branch_id),
    officer_id INT NOT NULL REFERENCES FieldOfficers(officer_id),
    name VARCHAR(100) NOT NULL,
    meeting_day VARCHAR(15) NOT NULL CHECK (meeting_day IN ('Saturday', 'Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday'))
);

CREATE TABLE Groups (
    group_id SERIAL PRIMARY KEY,
    center_id INT NOT NULL REFERENCES Centers(center_id),
    name VARCHAR(50) NOT NULL
);

CREATE TABLE Clients (
    client_id SERIAL PRIMARY KEY,
    group_id INT NOT NULL REFERENCES Groups(group_id),
    branch_id INT NOT NULL REFERENCES Branches(branch_id),
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    nid VARCHAR(20) NOT NULL UNIQUE,
    admission_date DATE DEFAULT CURRENT_DATE,
    status VARCHAR(20) DEFAULT 'Active' CHECK (status IN ('Active', 'Suspended', 'Closed'))
);

CREATE TABLE LoanProducts (
    product_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    interest_rate DECIMAL(5,2) NOT NULL CHECK (interest_rate >= 0.00),
    duration_months INT NOT NULL CHECK (duration_months > 0)
);
```

4.2 SQL Table Creation Script Continued & Seed Data (DML)

```

CREATE TABLE Loans (
    loan_id SERIAL PRIMARY KEY,
    client_id INT NOT NULL REFERENCES Clients(client_id),
    product_id INT NOT NULL REFERENCES LoanProducts(product_id),
    disbursed_amount DECIMAL(12,2) NOT NULL CHECK (disbursed_amount > 0.00),
    outstanding_balance DECIMAL(12,2) NOT NULL CHECK (outstanding_balance >= 0.00),
    disbursed_date DATE NOT NULL DEFAULT CURRENT_DATE,
    repayment_status VARCHAR(25) DEFAULT 'Active' CHECK (repayment_status IN ('Active', 'Paid',
'Defaulted', 'Written Off'))
);

CREATE TABLE RepaymentSchedule (
    schedule_id SERIAL PRIMARY KEY,
    loan_id INT NOT NULL REFERENCES Loans(loan_id),
    installment_no INT NOT NULL CHECK (installment_no > 0),
    due_date DATE NOT NULL,
    principal_due DECIMAL(12,2) NOT NULL CHECK (principal_due >= 0.00),
    interest_due DECIMAL(12,2) NOT NULL CHECK (interest_due >= 0.00),
    amount_paid DECIMAL(12,2) DEFAULT 0.00 CHECK (amount_paid >= 0.00),
    status VARCHAR(20) DEFAULT 'Unpaid' CHECK (status IN ('Unpaid', 'Partially Paid', 'Paid'))
);

CREATE TABLE Repayments (
    repayment_id SERIAL PRIMARY KEY,
    loan_id INT NOT NULL REFERENCES Loans(loan_id),
    amount DECIMAL(12,2) NOT NULL CHECK (amount > 0.00),
    payment_date DATE NOT NULL DEFAULT CURRENT_DATE,
    collected_by INT NOT NULL REFERENCES FieldOfficers(officer_id)
);

CREATE TABLE SavingsAccounts (
    savings_id SERIAL PRIMARY KEY,
    client_id INT NOT NULL UNIQUE REFERENCES Clients(client_id),
    balance DECIMAL(12,2) DEFAULT 0.00 CHECK (balance >= 0.00),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE SavingsTransactions (
    transaction_id SERIAL PRIMARY KEY,
    savings_id INT NOT NULL REFERENCES SavingsAccounts(savings_id),
    transaction_type VARCHAR(20) NOT NULL CHECK (transaction_type IN ('Deposit', 'Withdrawal')),
    amount DECIMAL(12,2) NOT NULL CHECK (amount > 0.00),
    transaction_date DATE NOT NULL DEFAULT CURRENT_DATE
);

```

Below is a snapshot of seed inserts for testing core flows:

```

INSERT INTO Branches (name, address) VALUES ('Savar Branch', 'Nabinagar, Savar, Dhaka');
INSERT INTO FieldOfficers (branch_id, first_name, last_name, hire_date) VALUES (1, 'Rahim', 'Uddin',
'2024-01-15');
INSERT INTO Centers (branch_id, officer_id, name, meeting_day) VALUES (1, 1, 'Bismillah Center',
'Tuesday');
INSERT INTO Groups (center_id, name) VALUES (1, 'Shapla Group');
INSERT INTO Clients (group_id, branch_id, first_name, last_name, nid) VALUES (1, 1, 'Fatema', 'Begum',
'1989261728394');
INSERT INTO SavingsAccounts (client_id, balance) VALUES (1, 1500.00);
INSERT INTO LoanProducts (name, interest_rate, duration_months) VALUES ('General Loan', 10.00, 12);
INSERT INTO Loans (client_id, product_id, disbursed_amount, outstanding_balance, disbursed_date) VALUES
(1, 1, 20000.00, 22000.00, '2026-01-06');

```

5. Advanced Database Concepts

5.1 Indexing Strategy and Query Optimization

In a production database with hundreds of thousands of borrowers and weekly transaction logs, index design is critical. Primary keys automatically receive B-tree indexes, but secondary and composite indexes must be strategically introduced to optimize performance:

1. **NID Index:** A B-tree index on the National ID field in the client table allows immediate lookup during registration to prevent duplicate fraud.

```
CREATE INDEX idx_clients_nid ON Clients(nid);
```

2. **Composite Loan Search Index:** Frequently, field officers list all active loans in their center. A composite index on `client_id` and `repayment_status` avoids sequential table scans.

```
CREATE INDEX idx_loans_client_status ON Loans(client_id, repayment_status);
```

3. **Transaction Search Index:** Sorting and aggregating repayments by date is common for monthly financial closures.

```
CREATE INDEX idx_repayments_date ON Repayments(payment_date);
```

5.2 Database Partitioning Strategy

As the MFI operates, the `Repayments` table grows exponentially, accumulating millions of rows annually. This slows down index scans. To solve this, range partitioning by date is implemented:



```
-- Partitioned Repayments Table DDL
CREATE TABLE Repayments_Partitioned (
    repayment_id INT,
    loan_id INT,
    amount DECIMAL(12,2),
    payment_date DATE NOT NULL,
    collected_by INT
) PARTITION BY RANGE (payment_date);

CREATE TABLE Repayments_2026_Q1 PARTITION OF Repayments_Partitioned
FOR VALUES FROM ('2026-01-01') TO ('2026-04-01');

CREATE TABLE Repayments_2026_Q2 PARTITION OF Repayments_Partitioned
FOR VALUES FROM ('2026-04-01') TO ('2026-07-01');
```

5.3 Triggers and Business Rules Enforcement

Triggers are critical to enforce transactional consistency and execute automated updates without relying on application-level code.

Trigger 1: Prevent Concurrent Active Loans

A strict policy in microfinance prohibits borrowers from holding more than one active loan at any given time. The trigger function below executes before a loan record is inserted, throwing an exception if a violation occurs.

```
CREATE OR REPLACE FUNCTION verify_active_loan_limit()
RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (
        SELECT 1 FROM Loans
        WHERE client_id = NEW.client_id
        AND repayment_status = 'Active'
    ) THEN
        RAISE EXCEPTION 'MFI Rule Violation: Client already holds an active loan account.';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_verify_active_loan_limit
BEFORE INSERT ON Loans
FOR EACH ROW
EXECUTE FUNCTION verify_active_loan_limit();
```

Trigger 2: Real-time Repayment and Loan Balance Update

When a field officer enters a repayment transaction, the system must automatically subtract the paid amount from the loan outstanding balance, and change the status to 'Paid' when fully cleared.

```
CREATE OR REPLACE FUNCTION sync_loan_balance_on_repayment()
RETURNS TRIGGER AS $$
BEGIN
    -- Update loan balance
    UPDATE Loans
    SET outstanding_balance = outstanding_balance - NEW.amount
    WHERE loan_id = NEW.loan_id;

    -- Update loan status to 'Paid' if outstanding reaches 0
    UPDATE Loans
    SET repayment_status = 'Paid', outstanding_balance = 0.00
    WHERE loan_id = NEW.loan_id AND outstanding_balance <= 0.00;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_sync_loan_balance
AFTER INSERT ON Repayments
FOR EACH ROW
EXECUTE FUNCTION sync_loan_balance_on_repayment();
```

5.4 Transaction Management and Concurrency Control

Microfinance collections require atomic transactions. When a client pays an installment, three events must happen atomically: recording the loan payment, marking the schedule row as paid, and depositing a mandatory saving amount. If any step fails, the entire transaction must roll back to prevent accounting discrepancies.

SQL Transaction Script (ACID Compliance)

```
BEGIN;

-- Step 1: Insert the collection payment
INSERT INTO Repayments (loan_id, amount, payment_date, collected_by)
VALUES (1, 500.00, '2026-06-16', 1);

-- Step 2: Update the specific schedule installment status
UPDATE RepaymentSchedule
SET amount_paid = amount_paid + 500.00,
    status = 'Paid',
    paid_date = '2026-06-16'
WHERE schedule_id = 45;

-- Step 3: Insert mandatory weekly savings deposit (BDT 100)
INSERT INTO SavingsTransactions (savings_id, transaction_type, amount, transaction_date)
VALUES (1, 'Deposit', 100.00, '2026-06-16');

-- Step 4: Update the client's savings account ledger balance
UPDATE SavingsAccounts
SET balance = balance + 100.00
WHERE savings_id = 1;

COMMIT;
```

Concurrency Control and Locking Strategy

During center meetings, multiple Field Officers synchronize their tablets simultaneously with the main office server. Under default Read Committed isolation, a race condition could occur if two processes try to record payments or check savings balances simultaneously.

To prevent lost updates, we enforce a row-level locking mechanism during savings withdrawals or balance checks. By appending `FOR UPDATE` to the select query, subsequent sessions attempting to lock the same row are forced to wait until the holding transaction commits or aborts:

```
SELECT balance FROM SavingsAccounts WHERE savings_id = 1 FOR UPDATE;
```

This locks the client's savings row exclusively, ensuring that concurrent deposits or withdrawals are serialized and ledger integrity is preserved.

6. Complex SQL Queries & Analytical Reporting

Operational reports and data-driven insights are critical for managing credit risks. This section presents complex SQL analytical scripts designed for the database.

6.1 Portfolio at Risk (PAR) Analysis

The PAR metric measures the outstanding balance of all active loans that have unpaid installments past their due date, divided by the MFI's total outstanding portfolio. It is the primary indicator of credit risk. The query below calculates **PAR-30** (repayments overdue by more than 30 days):

```
WITH OverdueLoans AS (
    SELECT DISTINCT s.loan_id
    FROM RepaymentSchedule s
    WHERE s.status IN ('Unpaid', 'Partially Paid')
    AND s.due_date < CURRENT_DATE - INTERVAL '30 days'
),
PAR_Stats AS (
    SELECT
        SUM(CASE WHEN l.loan_id IN (SELECT loan_id FROM OverdueLoans)
            THEN l.outstanding_balance ELSE 0 END) as overdue_portfolio,
        SUM(l.outstanding_balance) as total_portfolio
    FROM Loans l
    WHERE l.repayment_status = 'Active'
)
SELECT
    overdue_portfolio,
    total_portfolio,
    ROUND((overdue_portfolio / total_portfolio) * 100, 2) AS par_30_percentage
FROM PAR_Stats;
```

6.2 Collection Efficiency Rate (CER) per Branch

The Collection Efficiency Rate measures the ratio of the total amount collected at a branch to the total amount that was scheduled to be collected within a given month (e.g., June 2026):

```
SELECT
    b.branch_id,
    b.name AS branch_name,
    SUM(r.amount) AS actual_collected,
    (SELECT SUM(principal_due + interest_due)
     FROM RepaymentSchedule rs
     JOIN Loans l ON rs.loan_id = l.loan_id
     JOIN Clients c ON l.client_id = c.client_id
     WHERE c.branch_id = b.branch_id
     AND rs.due_date BETWEEN '2026-06-01' AND '2026-06-30') AS scheduled_amount,
    ROUND((SUM(r.amount) /
        (SELECT SUM(principal_due + interest_due)
         FROM RepaymentSchedule rs
         JOIN Loans l ON rs.loan_id = l.loan_id
         JOIN Clients c ON l.client_id = c.client_id
         WHERE c.branch_id = b.branch_id
         AND rs.due_date BETWEEN '2026-06-01' AND '2026-06-30')) * 100, 2) AS cer_percentage
FROM Repayments r
JOIN FieldOfficers f ON r.collected_by = f.officer_id
JOIN Branches b ON f.branch_id = b.branch_id
WHERE r.payment_date BETWEEN '2026-06-01' AND '2026-06-30'
GROUP BY b.branch_id, b.name;
```

6.3 Top Field Officers Performance Ranking

Using a window function (`DENSE_RANK`), field officers are ranked within their branches based on their collection recovery volume:

```
WITH OfficerCollection AS (
    SELECT f.branch_id, f.officer_id, f.first_name || ' ' || f.last_name AS officer_name, SUM(r.amount)
    AS total_collected
    FROM Repayments r JOIN FieldOfficers f ON r.collected_by = f.officer_id GROUP BY f.branch_id,
    f.officer_id
)
SELECT branch_id, officer_name, total_collected,
       DENSE_RANK() OVER (PARTITION BY branch_id ORDER BY total_collected DESC) AS ranking
FROM OfficerCollection;
```

6.4 Client Savings vs. Outstanding Loan Balance Summary

An critical operational check is matching client savings against their outstanding loan balance to analyze internal capital ratios:

```
SELECT c.client_id, c.first_name || ' ' || c.last_name AS client_name,
       COALESCE(sa.balance, 0.00) AS total_savings, COALESCE(SUM(l.outstanding_balance), 0.00) AS
total_outstanding_loan,
       CASE WHEN COALESCE(sa.balance, 0.00) >= COALESCE(SUM(l.outstanding_balance), 0.00) THEN 'Fully
Covered' ELSE 'Deficit' END AS coverage_status
FROM Clients c LEFT JOIN SavingsAccounts sa ON c.client_id = sa.client_id
LEFT JOIN Loans l ON c.client_id = l.client_id AND l.repayment_status = 'Active' GROUP BY c.client_id,
sa.balance;
```

6.5 Monthly Disbursement and Recovery Trend Analysis

This query outputs month-on-month growth trends in loans disbursed versus recovery volume to assess liquidity:

```
WITH MonthlyDisbursement AS (
    SELECT DATE_TRUNC('month', disbursed_date) AS month, SUM(disbursed_amount) AS total_disbursed
    FROM Loans GROUP BY DATE_TRUNC('month', disbursed_date)
),
MonthlyRecovery AS (
    SELECT DATE_TRUNC('month', payment_date) AS month, SUM(amount) AS total_recovered
    FROM Repayments GROUP BY DATE_TRUNC('month', payment_date)
)
SELECT TO_CHAR(COALESCE(d.month, r.month), 'YYYY-MM') AS report_month, COALESCE(d.total_disbursed,
0.00) AS disbursement,
       COALESCE(r.total_recovered, 0.00) AS recovery, COALESCE(r.total_recovered, 0.00) -
COALESCE(d.total_disbursed, 0.00) AS net_cash_flow
FROM MonthlyDisbursement d FULL OUTER JOIN MonthlyRecovery r ON d.month = r.month ORDER BY
report_month;
```

7. Limitations & Discussion

While the proposed database solution addresses key operational pain points, it has limitations. First, network connectivity in deep rural areas of Bangladesh remains intermittent. Since this database design is centralized, field officers require offline synchronization capabilities in their mobile applications to collect data locally and write-back when connected, which requires conflict-resolution protocols.

Second, security challenges such as identity fraud demand biometric database fields (e.g., storing fingerprint hashes). Role-Based Access Control (RBAC) needs to be strictly implemented at the database level to prevent database administrators or staff from altering repayment logs retrospectively.

8. Conclusion & Recommendations

Transitioning microfinance institutions from paper-based bookkeeping to a relational database architecture is essential for scalable growth. The designed system organizes data into a 3NF normalized schema, ensuring zero redundancies and high performance. Using SQL constraints, indexes, and triggers, critical microfinance policies (e.g., limiting a client to one active loan, or keeping savings balances accurate) are locked directly into the storage engine.

It is recommended that MFIs: (1) integrate the database with SMS and Mobile Financial Services (MFS) like bKash or Nagad to automate receipts, (2) transition from a single server to distributed read-replicas to handle heavy branch workloads, and (3) establish regular database replication routines for disaster recovery.

9. References (APA Style)

- Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.
- Gupta, A., & Chaudhri, S. (2017). Optimization of relational queries in data warehousing environments. *IEEE Transactions on Knowledge and Data Engineering*, 29(4), 812-825.
- Microcredit Regulatory Authority (MRA). (2024). *Annual MFI Statistics Report of Bangladesh*. MRA Publications.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill.
- Yunus, M. (2007). *Banker to the Poor: Micro-lending and the battle against world poverty*. PublicAffairs.